Curriculum für Certified Professional for Software Architecture (CPSA)®

Foundation Level

2027.1-RC1-DE-20261113



Lizenz und Copyright

© (Copyright), International Software Architecture Qualification Board e. V. (iSAQB® e. V.) 2026

Die Nutzung des Lehrplans ist nur unter den nachfolgenden Voraussetzungen erlaubt:

1. Sie möchten das Zertifikat zum CPSA Certified Professional for Software Architecture Foundation Level® oder CPSA Certified Professional for Software Architecture Advanced Level® erwerben. Für den Erwerb des Zertifikats ist es gestattet, die Text-Dokumente und/oder Lehrpläne zu nutzen, indem eine Arbeitskopie für den eigenen Rechner erstellt wird. Soll eine darüber hinausgehende Nutzung der Dokumente und/oder Lehrpläne erfolgen, zum Beispiel zur Weiterverbreitung an Dritte, Werbung etc., bitte unter info@isaqb.org nachfragen. Es müsste dann ein eigener Lizenzvertrag geschlossen werden.
2. Sind Sie Trainer oder Trainingsprovider, ist die Nutzung der Dokumente und/oder Lehrpläne nach Erwerb einer Nutzungslizenz möglich. Hierzu bitte unter info@isaqb.org nachfragen. Lizenzverträge, die alles umfassend regeln, sind vorhanden.
3. Falls Sie weder unter die Kategorie 1. noch unter die Kategorie 2. fallen, aber dennoch die Dokumente und/oder Lehrpläne nutzen möchten, nehmen Sie bitte ebenfalls Kontakt unter info@isaqb.org zum iSAQB e. V. auf. Sie werden dort über die Möglichkeit des Erwerbs entsprechender Lizenzen im Rahmen der vorhandenen Lizenzverträge informiert und können die gewünschten Nutzungsgenehmigungen erhalten.

Wichtiger Hinweis

Grundsätzlich weisen wir darauf hin, dass dieser Lehrplan urheberrechtlich geschützt ist. Alle Rechte an diesen Copyrights stehen ausschließlich dem International Software Architecture Qualification Board e. V. (iSAQB® e. V.) zu.

Die Abkürzung "e. V." ist Teil des offiziellen Namens des iSAQB und steht für "eingetragener Verein", der seinen Status als juristische Person nach deutschem Recht beschreibt. Der Einfachheit halber wird iSAQB e. V. im Folgenden ohne die Verwendung dieser Abkürzung als iSAQB bezeichnet.

Inhaltsverzeichnis

Lizenz und Copyright . . . ii
Lernziele im Überblick . . . iv
Einleitung . . . 1
Was dieser Lehrplan enthält . . . 1
Was vermittelt eine Foundation-Level-Schulung? . . . 1
Abgrenzung . . . 2
Voraussetzungen . . . 3
Struktur, Dauer, Didaktik . . . 4
Lernziele und Prüfungsrelevanz . . . 5
Aktuelle Version sowie öffentliches Repository . . . 5
1. Grundbegriffe von Softwarearchitektur . . . 6
Zielsetzung . . . 6
Wesentliche Begriffe . . . 6
Lernziele . . . 6
2. Anforderungen und Randbedingungen . . . 9
Zielsetzung . . . 9
Wesentliche Begriffe . . . 9
Lernziele . . . 9
3. Entwurf und Entwicklung von Softwarearchitekturen . . . 13
Zielsetzung . . . 13
Wesentliche Begriffe . . . 13
Lernziele . . . 13
4. Beschreibung und Kommunikation von Softwarearchitekturen . . . 20
Zielsetzung . . . 20
Wesentliche Begriffe . . . 20
Lernziele . . . 20
5. Einsatz und Risiken von Künstlicher Intelligenz . . . 24
Zielsetzung . . . 24
Wesentliche Begriffe . . . 24
Lernziele . . . 24
6. Beispiele für Softwarearchitekturen . . . 26
Lernziele . . . 26
Referenzen . . . 27

Lernziele im Überblick

- LZ 01-01: Definitionen von Softwarearchitektur verstehen (R1)
- LZ 01-02: Ziele und Nutzen von Softwarearchitektur verstehen und erläutern (R1)
- LZ 01-03: Langfristige Auswirkungen von Softwarearchitektur kennen (R3)
- LZ 01-04: Aufgaben und Verantwortung von Softwarearchitekt:innen verstehen (R1)
- LZ 01-05 This LG is temporarily vacant
- LZ 01-06: Rolle von Softwarearchitekt:innen mit anderen Stakeholdern in Beziehung setzen (R1)
- LZ 01-07: Bedeutung von Daten und Datenmodellen (R2)
- LZ 02-01: Stakeholder-Anliegen verstehen (R1, R3)
- LZ 02-02: Anforderungen und Randbedingungen klären und berücksichtigen können (R1-R3)
- LZ 02-03: Qualitäten eines Softwaresystems verstehen und erklären (R1)
- LZ 02-04: Anforderungen an Qualitäten formulieren (R1-R3)
- LZ 02-05: Explizite Aussagen vor impliziten Annahmen bevorzugen (R1, R2)
- LZ 03-01: Anforderungen durch Architektur erreichen (R1)
- LZ 03-02: Softwarearchitekturen entwerfen (R1)
- LZ 03-03: Vorgehen und Heuristiken zur Architekturentwicklung auswählen und anwenden können (R1,R3)
- LZ 03-04: Entwurfsprinzipien erläutern und anwenden (R1-R3)
- LZ 03-05: Zusammenhang zwischen Feedback-Schleifen und Risiko (R1, R2)
- LZ 03-06: Abhängigkeiten von Bausteinen managen (R1)
- LZ 03-07: Schnittstellen entwerfen und spezifizieren (R1-R3)
- LZ 03-08: Wichtige Muster beschreiben, erklären und angemessen anwenden (R1, R3)
- LZ 03-09: Architektur analysieren (R1)
- LZ 03-10: Querschnittsthemen identifizieren und Querschnittskonzepte entwerfen und umsetzen (R1)
- LZ 03-11: Grundlegende Prinzipien von Software-Deployments kennen (R3)
- LZ 03-12: Technische und geschäftliche Sichten verwenden können (R3)
- LZ 04-01: Anforderungen an technische Dokumentation erläutern und berücksichtigen (R1)
- LZ 04-02: Softwarearchitekturen beschreiben und kommunizieren (R1-R3)
- LZ 04-03: Notations-/Modellierungsmittel für Beschreibung von Softwarearchitektur erläutern und anwenden (R2-R3)
- LZ 04-04: Lernziel nicht gefunden (R3)
- LZ 04-05: Architektursichten erläutern und anwenden (R1)
- LZ 04-06: Schnittstellen dokumentieren (R1)
- LZ 04-07: Querschnittsthemen dokumentieren und kommunizieren (R2)
- LZ 04-08: Architekturentscheidungen erläutern und dokumentieren (R1-R2)

- LZ 04-09: Weitere Hilfsmittel und Werkzeuge zur Dokumentation kennen (R2)
- LZ 05-01: Über den Einsatz von KI in der Architekturarbeit entscheiden (R1, R2)
- LZ 05-02: Verifikation von KI-generierten Artefakten (R1, R2)
- LZ 05-03: Risiken, ethische und gesellschaftliche Externalitäten erkennen (R2, R3)
- LZ 06-01: Bezug von Anforderungen und Randbedingungen zur Lösung erfassen (R3)
- LZ 06-02: Technische Umsetzung einer Lösung nachvollziehen (R3)

Einleitung

Was dieser Lehrplan enthält

Dieser Lehrplan für den *Certified Professional for Software Architecture - Foundation Level* (CPSA-F) beinhaltet die Lernziele, die man beherrschen sollte, um die Rolle Softwarearchitekt:in zu übernehmen.

Seine Struktur orientiert sich an den grundlegenden Aktivitäten und Verantwortlichkeiten der Softwarearchitektur als Rolle:

- Anforderungen und Randbedingungen klären
- Entwurf und Entwicklung von Softwarearchitekturen, dabei strukturelle und konzeptionelle Entscheidungen treffen
- Beschreiben und Kommunizieren von Softwarearchitekturen für verschiedene Stakeholder
- Analysieren und Bewerten von Softwarearchitekturen

Was vermittelt eine Foundation-Level-Schulung?

Lizenzierte Schulungen zum *Certified Professional for Software Architecture – Foundation Level* (CPSA-F) vermitteln grundlegende Kenntnisse und Fertigkeiten für den Entwurf einer angemessenen Softwarearchitektur für kleine und mittlere IT-Systeme. Die Teilnehmenden erweitern und vertiefen ihre bestehenden Erfahrungen und Fähigkeiten in der Softwareentwicklung, um relevante Vorgehensweisen, Methoden und Prinzipien für die Entwicklung von Softwarearchitekturen. Durch das Gelernte können sie auf Grundlage angemessen detaillierter Anforderungen und Randbedingungen eine adäquate Softwarearchitektur entwerfen, kommunizieren, analysieren, bewerten und weiterentwickeln. Schulungen zum CPSA-F vermitteln Grundlagenwissen unabhängig von spezifischen Entwurfsmethoden, Vorgehensmodellen, Programmiersprachen oder Werkzeugen. Dadurch können die Teilnehmenden ihre erworbene Fertigkeiten auf ein breites Spektrum von Einsatzfällen anwenden.

Im Mittelpunkt steht der Erwerb folgender Fähigkeiten:

- mit anderen Beteiligten aus den Bereichen Anforderungsmanagement, Projektmanagement, Entwicklung und Test wesentliche Architekturentscheidungen abzustimmen
- die wesentlichen Schritte beim Entwurf von Softwarearchitekturen zu verstehen sowie für kleine und mittlere Systeme selbstständig durchzuführen
- Softwarearchitekturen auf Basis von Sichten, Mustern und technischen Konzepten zu dokumentieren und zu kommunizieren.

Weiterhin behandeln CPSA-F-Schulungen:

- den Begriff und die Bedeutung von Softwarearchitektur
- die Aufgaben und Verantwortung von Softwarearchitekten
- die Rolle von Softwarearchitekt:innen in Entwicklungsvorhaben
- State-of-the-Art-Methoden und -Praktiken zur Entwicklung von Softwarearchitekturen.

Abgrenzung

Dieser Lehrplan reflektiert den aus heutiger Sicht des iSAQB e.V. notwendigen und sinnvollen Inhalt zur Erreichung der Lernziele des CPSA-F. Er stellt keine vollständige Beschreibung des Wissensgebiets „Softwarearchitektur“ dar.

Folgende Themen oder Konzepte sind **nicht Bestandteil des CPSA-F**:

- konkrete Implementierungstechnologien, -frameworks oder -bibliotheken
- Programmierung oder Programmiersprachen
- Spezifische Vorgehensmodelle
- Grundlagen oder Notationen der Modellierung (wie etwa UML)
- Systemanalyse und Requirements Engineering (siehe dazu das Ausbildungs- und Zertifizierungsprogramm des IREB e. V., https://ireb.org, International Requirements Engineering Board)
- Test (siehe dazu das Ausbildungs- und Zertifizierungsprogramm des ISTQB e. V., https://istqb.org, International Software Testing Qualification Board)
- Projekt- oder Produktmanagement
- Einführung in spezifische Werkzeuge.

Ziel des Trainings ist es, die Grundlagen für den Erwerb der für den jeweiligen Einsatzfall notwendigen weiterführenden Kenntnisse und Fertigkeiten zu vermitteln.

Voraussetzungen

Der iSAQB e. V. kann in Zertifizierungsprüfungen die hier genannten Voraussetzungen durch entsprechende Fragen prüfen.

Teilnehmende sollten die im Nachfolgenden genannten Kenntnisse und/oder Erfahrung mitbringen. Insbesondere bilden substanzielle praktischen Erfahrungen aus der Softwareentwicklung im Team eine wichtige Voraussetzung zum Verständnis des vermittelten Lernstoffes und für eine erfolgreiche Zertifizierung.

- mehr als 18 Monate praktische Erfahrung in arbeitsteiliger Softwareentwicklung (d.h. in Teams), erworben durch Programmierung unterschiedlicher Systeme außerhalb der Ausbildung
- Kenntnisse und praktische Erfahrung in mindestens einer höheren Programmiersprache, insbesondere:
  - Konzepte der
    - Modularisierung (Pakete, Namensräume)
    - Parameterübergabe (*Call-by-Value, Call-by-Reference*)
    - Gültigkeit (*scope*), beispielsweise von Typ- oder Variablendeklaration und -definition
  - Grundlagen von Typsystemen (statische und dynamische Typisierung, generische Datentypen)
  - Fehler- und Ausnahmebehandlung in Software
  - Mögliche Probleme von globalem Zustand und globalen Variablen
- Grundlegende Kenntnisse von:
  - Modellierung und Abstraktion
  - Algorithmen und Datenstrukturen (etwa Listen, Bäume, HashTable, Dictionary/Map)
  - UML (Klassen-, Paket-, Komponenten- und Sequenzdiagramme) und deren Bezug zum Quellcode
  - Vorgehen beim Testen von Software (z. B. Unit- und Akzeptanztests)

Hilfreich für das Verständnis einiger Konzepte sind darüber hinaus:

- Grundbegriffe bzw. Unterschiede von imperativer, deklarativer, objektorientierter und funktionaler Programmierung
- praktische Erfahrung in
  - einer höheren Programmiersprache
  - Konzeption und Implementierung verteilt ablaufender Anwendungen, wie etwa Client/Server-Systeme oder Web-Anwendungen
  - technischer Dokumentation, insbesondere in der Dokumentation von Quellcode, Systementwürfen oder technischen Konzepten

Struktur, Dauer, Didaktik

Die in den nachfolgenden Kapiteln des Lehrplans genannten Zeiten sind lediglich Empfehlungen. Die Dauer einer Schulung sollte mindestens 3 Tage betragen, kann aber durchaus länger sein. Anbieter können sich durch Dauer, Didaktik, Art und Aufbau der Übungen sowie der detaillierten Kursgliederung voneinander unterscheiden. Insbesondere die Art (fachliche und technische Domänen) der Beispiele und Übungen können die jeweiligen Schulungsanbieter individuell festlegen.

Inhalt	Empfohlene Dauer (min)
1. Grundlagen	120
2. Anforderungen und Randbedingungen	180
3. Entwurf und Entwicklung	360
4. Beschreibung und Kommunikation	240
5. Einsatz und Risiken von Künstlicher Intelligenz	60
6. Beispiele	90
Summe	1050

Die Summe der empfohlenen Zeiten (1050 Minuten bzw. 17,5 Stunden) entspricht nahezu der Netto-Lehrzeit einer dreitägigen Schulung (3 Tage à 4 × 90 Minuten = 1080 Minuten). Pausen, über Curriculum-Inhalte hinausgehende Diskussionen und Pufferzeiten sind darin nicht enthalten und müssen von Schulungsanbietern zusätzlich eingeplant werden.

Lernziele und Prüfungsrelevanz

Die Kapitel des Lehrplans sind anhand von priorisierten Lernzielen gegliedert. Die Prüfungsrelevanz dieser Lernziele beziehungsweise deren Unterpunkte ist beim jeweiligen Lernziel ausdrücklich gekennzeichnet (durch Angabe der Kennzeichen R1, R2 oder R3, siehe nachstehende Tabelle).

Jedes Lernziel beschreibt die zu vermittelnden Inhalte inklusive ihrer Kernbegriffe und -konzepte. Bezüglich der Prüfungsrelevanz verwendet der Lehrplan folgende Kategorien:

ID	Lernziel-Kategorie	Bedeutung	Relevanz für Prüfung
R1	Können	Diese Inhalte sollen die Teilnehmenden nach der Schulung selbstständig anwenden können. Innerhalb der Schulung sollten diese Inhalte durch Übungen und Diskussionen abgedeckt werden.	Inhalte **werden** geprüft.
R2	Verstehen	Diese Inhalte sollen die Teilnehmenden grundsätzlich verstehen. Sie werden in Schulungen i. d. R. nicht durch Übungen vertieft.	Inhalte **können** geprüft werden.
R3	Kennen	Diese Inhalte (Begriffe, Konzepte, Methoden, Praktiken oder Ähnliches) können das Verständnis unterstützen oder das Thema motivieren. Sie werden in Schulungen bei Bedarf thematisiert.	Inhalte **werden nicht** geprüft.

Bei Bedarf enthalten die Lernziele Verweise auf weiterführende Literatur, Standards oder andere Quellen. Die Abschnitte "Begriffe und Konzepte" zu Beginn jedes Kapitels zeigen Worte, die mit dem Inhalt des Kapitels in Verbindung stehen und z. T. auch in den Lernzielen verwendet werden.

Aktuelle Version sowie öffentliches Repository

Den aktuellen Stand dieses Dokumentes finden Sie auf der offiziellen Download-Seite unter https://public.isaqb.org/.

Das Dokument wird in einem öffentlichen Repository unter https://github.com/isaqb-org/curriculum-foundation gepflegt, alle Änderungen sind dort sichtbar.

Bitte melden Sie eventuelle Probleme in unserem öffentlichen Issue Tracker bei https://github.com/isaqb-org/curriculum-foundation/issues.

1. Grundbegriffe von Softwarearchitektur

Dauer: 120 Min.	Übungszeit: Keine

Zielsetzung

Ziel dieses Abschnitts ist es, den Teilnehmenden ein grundlegendes Verständnis der wichtigsten Begriffe und Konzepte von Softwarearchitektur zu vermitteln. Sie lernen verschiedene Definitionen und deren Gemeinsamkeiten kennen, verstehen die wesentlichen Ziele und Vorteile von Softwarearchitektur und können diese anderen Stakeholdern vermitteln. Weiterhin können sie die wichtigsten Aufgaben und Verantwortlichkeiten von Softwarearchitektinnen und Softwarearchitekten benennen und erklären. Außerdem wird deren Rolle im organisatorischen Kontext und ihre Interaktion mit anderen Stakeholdern thematisiert, um es den Teilnehmenden zu ermöglichen, einen effektiven Beitrag zu verschiedenartigen Softwareentwicklungsprojekten zu leisten.

Wesentliche Begriffe

Softwarearchitektur; Architekturdomänen; Struktur; Bausteine; Komponenten; Schnittstellen; Beziehungen; Querschnittsthemen; Nutzen von Softwarearchitektur; Softwarearchitekt:innen und deren Verantwortlichkeiten; Rolle; Aufgaben und benötigte Fähigkeiten; Stakeholder und deren Anliegen; Anforderungen; Randbedingungen; Einflussfaktoren

Lernziele

LZ 01-01: Definitionen von Softwarearchitektur verstehen (R1)

Softwarearchitekt:innen kennen und verstehen die Gemeinsamkeiten vieler Definitionen von Softwarearchitektur:

- Komponenten/Bausteine mit Schnittstellen und Beziehungen
- Bausteine als allgemeiner Begriff, Komponenten als eine spezielle Ausprägung davon
- Strukturen, Querschnittsthemen, Prinzipien
- Architekturentscheidungen und ihre Auswirkungen auf das gesamte System und seinen Lebenszyklus

Referenzen

[Bass+2021], [Kruchten 2004], [Starke+2023a]

LZ 01-02: Ziele und Nutzen von Softwarearchitektur verstehen und erläutern (R1)

Softwarearchitekt:innen können die folgenden wesentlichen Ziele und Nutzen der Softwarearchitektur begründen:

- Entwurf, Implementierung, Pflege und Betrieb von Systemen zu unterstützen
- funktionale Anforderungen zu erreichen bzw. deren Erfüllbarkeit sicherzustellen
- Anforderungen wie Zuverlässigkeit, Wartbarkeit, Änderbarkeit, Sicherheit, Energieeffizienz zu erreichen
- Verständnis für Strukturen und Konzepte des Systems bei allen relevanten Stakeholdern sicherstellen
- Komplexität systematisch zu reduzieren
- architekturrelevante Richtlinien für Implementierung und Betrieb zu spezifizieren

LZ 01-03: Langfristige Auswirkungen von Softwarearchitektur kennen (R3)

Softwarearchitekt:innen kennen

- den Zusammenhang zwischen Architekturentscheidungen und der zukünftigen Anpassbarkeit und Wartbarkeit des Systems
- die Auswirkungen durch Änderungen von Anforderungen, Technologien oder Systemumgebung auf bestehende Architekturentscheidungen
- die langfristigen Konsequenzen von Architekturentscheidungen auf verschiedene Qualitätsmerkmale des Systems
- die Wechselwirkungen zwischen IT-Systemen und den unterstützten Geschäfts- und Betriebsprozessen

Sie wissen, wie sie die Auswirkungen von Architekturentscheidungen auf die langfristige Entwicklung eines Systems analysieren können.

Referenzen

[Bass+2021], [Lehman 1980], [Wiki-LehmansLaws], [Lilienthal 2024], [Ford+2023], [Rajlich+2000], [Richards+2020]

LZ 01-04: Aufgaben und Verantwortung von Softwarearchitekt:innen verstehen (R1)

Softwarearchitekt:innen tragen die Verantwortung für die Erreichung der Anforderungen und die Entwicklung der Architektur der Lösung. Sie müssen diese Verantwortung, abhängig vom jeweiligen Prozess- oder Vorgehensmodell, mit der Gesamtverantwortung der Projektleitung oder anderen Rollen koordinieren.

Aufgaben und Verantwortung von Softwarearchitekt:innen:

- Anforderungen und Randbedingungen klären und hinterfragen. Notwendige Verfeinerungen mit den entsprechenden Stakeholdern koordinieren und abstimmen.
- Strukturentscheidungen hinsichtlich Systemzerlegung und Bausteinstruktur treffen, dabei Abhängigkeiten und Schnittstellen zwischen den Bausteinen festlegen
- Querschnittsthemen entscheiden (beispielsweise Persistenz, Kommunikation, GUI)
- Softwarearchitektur auf Basis von Sichten, Mustern sowie technischen und Querschnittsthemen kommunizieren und dokumentieren
- Umsetzung und Implementierung der Architektur begleiten, Rückmeldungen der beteiligten Stakeholder bei Bedarf in die Architektur einarbeiten, Konsistenz von Quellcode und Softwarearchitektur prüfen und sicherstellen
- Softwarearchitektur analysieren und bewerten, insbesondere hinsichtlich Risiken bezüglich der Erreichung von Anforderungen.
- Die Konsequenzen von Architekturentscheidungen erkennen, aufzeigen und gegenüber anderen Stakeholdern argumentieren

Sie sollen selbstständig die Notwendigkeit von Iterationen bei allen Aufgaben erkennen und Möglichkeiten für entsprechende Rückmeldung aufzeigen.

LZ 01-05 This LG is temporarily vacant

LZ 01-06: Rolle von Softwarearchitekt:innen mit anderen Stakeholdern in Beziehung setzen (R1)

Softwarearchitekt:innen können ihre Rolle erklären. Sie sollten ihren Beitrag zur Systementwicklung in Verbindung mit anderen Stakeholdern und Organisationseinheiten kontextspezifisch ausgestalten, insbesondere zu:

- Produktmanagement, Product-Owner
- Projektleitung und -management
- Anforderungsanalytiker:innen (System-/Businessanalyse, Anforderungsmanagement, Fachbereich)
- Entwicklung
- Qualitätssicherung und Test
- IT-Betrieb (Produktion, Rechenzentren), zutreffend primär für Informationssysteme
- Hardwareentwicklung und Systemarchitektur, zutreffend primär für eingebettete und hardwarenahe Systeme
- Unternehmensarchitektur, Architekturboard.

LZ 01-07: Bedeutung von Daten und Datenmodellen (R2)

Softwarearchitekt:innen verstehen die Bedeutung von Daten und Datenmodellen (unabhängig von ihrer physischen Repräsentation) für die Architektur. Sie

- können Datenmodelle identifizieren, die maßgeblichen Einfluss auf die Architektur haben.
- können solche Datenmodelle systematisch entwerfen.
- verstehen den Unterschied zwischen Produkten und Summen in der Datenmodellierung.

Softwarearchitekt:innen

- verstehen die Bedeutung der Entkopplung von Datenmodellen und ihrer Repräsentation in Datenbanken, Dateien und Übertragungsprotokollen.
- können die Auswirkungen von Daten auf Architekturentscheidungen z.B. in Bezug auf Speicherung, Sicherheit, Skalierbarkeit, Zuverlässigkeit, Performance usw. erläutern.

Referenzen

[Felleisen+2014], [Sperber+2023], [Sperber+2024], [Kleppmann 2017], [Ford+2021]

2. Anforderungen und Randbedingungen

Dauer: 90 Min.	Übungszeit: 90 Min.

Zielsetzung

Dieser Abschnitt vertieft das Verständnis der Teilnehmenden für Stakeholder-Anliegen, Anforderungen und Qualitäten von Softwaresystemen. Sie lernen, den Einfluss von Stakeholdern auf Architekturentscheidungen zu erkennen sowie Konflikte und Synergien im Kontext von Entwicklungsprojekten einzuschätzen. Durch die Auseinandersetzung mit verschiedenen Anforderungen und Einschränkungen erhalten sie einen Einblick in die effektive Berücksichtigung der Bedürfnisse der Stakeholder und der Projektvorgaben. Weiterhin erkennen sie die Bedeutung von Qualitäten eines Softwaresystems als entscheidende Faktoren für den Architekturentwurf. Sie können solche Anforderungen mit Hilfe von Szenarien formulieren.

Wesentliche Begriffe

Qualität; Qualitätsmerkmale; DIN/ISO 25010; Q42; Qualitätsszenarien; Kompromisse/Wechselwirkungen von Qualitätseigenschaften; Anforderungen; Randbedingungen; Stakeholder-Anliegen

Lernziele

LZ 02-01: Stakeholder-Anliegen verstehen (R1, R3)

Architekten können Stakeholder und deren Anliegen sowie deren Auswirkungen auf die Softwarearchitektur oder den Entwurfs- und Entwicklungsprozess identifizieren. (R1)

Beispiele für Stakeholder und ihre Anliegen (R3):

Interessengruppe	Anliegen der Interessengruppe
Produktmanagement	z. B. benötigte Zeit für die Umsetzung der Anforderungen
Entwicklungsteam	z. B. zu implementierende Komponenten und Schnittstellen, Protokolle, technische Anforderungen und Randbedingungen
Requirements Engineering, Product-Owner, Business-Analyse, Fachbereiche	z. B. Erfüllung der Anforderungen
Projektmanagement	z. B. benötigte Zeit und Budget für die Umsetzung, verbundene Risiken des gewählten architekturellen Ansatzes
Qualitätssicherung und Test	z. B. isoliertes Testen von Komponenten
Betrieb	z. B. Infrastrukturanforderungen im Zusammenhang mit dem Betrieb des Systems

Softwarearchitekt:innen können potenzielle Konflikte zwischen kurz- und langfristigen Zielen identifizieren (z.B. Geschäfts- und Projektziele vs. Architektur- und Wartbarkeitsziele). Sie verstehen, dass sie die relevanten Stakeholder einbeziehen müssen, um diese Konflikte zu lösen. (R1)

Architekt:innen verstehen, dass nicht alle Anliegen der Stakeholder in Anforderungen umgesetzt werden können oder werden, aber dennoch berücksichtigt werden müssen. (R3)

Architekt:innen können die Anliegen der Stakeholder nutzen, um fehlende oder widersprüchliche Anforderungen zu entdecken und/oder Anforderungen und Einschränkungen an der Architektur zu validieren, z.B. in Stakeholder-Interviews. (R3)

LZ 02-02: Anforderungen und Randbedingungen klären und berücksichtigen können (R1-R3)

Softwarearchitekt:innen verstehen, dass sowohl Anforderungen als auch Randbedingungen Auswirkungen auf die Architektur und die Architekturarbeit haben können (R2). Sie sind in der Lage, Anforderungen und Randbedingungen zu klären und beim Architekturentwurf und im Entwicklungsprozess zu berücksichtigen. Sie verstehen, dass ihre Entscheidungen zu zusätzlichen Anforderungen führen oder Änderungen an bestehenden Anforderungen erforderlich machen können.

Sie erkennen und berücksichtigen den Einfluss von:

- produktbezogenen Anforderungen wie (R1)
  - funktionale Anforderungen
  - Qualitätsanforderungen
- Technologische Randbedingungen wie
  - bestehende oder geplante Hardware- und Software-Infrastruktur (R1)
  - technologische Beschränkungen für Datenstrukturen und Schnittstellen (R2)
  - Referenzarchitekturen, Bibliotheken, Komponenten und Frameworks (R1)
  - Programmiersprachen (R2)
- Organisatorischen Randbedingungen wie
  - Organisationsstruktur von Entwicklungsteams und Auftraggebenden (R1), insbesondere das Gesetz von Conway (R2)
  - Unternehmens- und Teamkultur (R3)
  - Partnerschaften und Kooperationen (R2)
  - Normen, Richtlinien und Prozessmodelle (z. B. Genehmigungs- und Freigabeprozesse) (R2)
  - Verfügbarkeit von Ressourcen wie Budget, Zeit und Personal (R1)
  - Verfügbarkeit, Qualifikation und Engagement von Mitarbeitenden (R1)
- Regulatorischen Randbedingungen wie (R2)
  - lokale und internationale rechtliche Einschränkungen
  - Vertrags- und Haftungsfragen
  - Datenschutzgesetze und Gesetze zum Schutz der Privatsphäre
  - Fragen der Einhaltung oder Verpflichtungen zur Beweislast
- Trends wie (R3)
  - Markttrends
  - Technologietrends (z. B. Cloud, Microservices, Container, generative KI oder LLMs)
  - Methodik-Trends (z. B. Agilität)

Softwarearchitekt:innen können beschreiben, wie diese Einflüsse auf Architekturentscheidungen wirken, und sie können die Konsequenzen einer Änderung der Einflussfaktoren anhand von Beispielen erläutern

(R2).

Referenzen

[IREB Foundation], [Bass+2021], [Gharbi+2024], [Starke 2024], [Richards+2020], [Pohl 2025]

LZ 02-03: Qualitäten eines Softwaresystems verstehen und erklären (R1)

Softwarearchitekt:innen wissen dass der Begriff "Qualität" in verschiedenen Kontexten unterschiedlich benutzt wird:

- im Kontext von Qualitätsmanagement im Sinne von "Güte" und
- im Sinne spezifischer Eigenschaften (eines Softwaresystems).

In diesem Lernziel geht es um letzteres.

Softwarearchitekt:innen können erklären dass:

- es unterschiedliche Taxonomien gibt, die Qualitäten von Softwaresystemen kategorisieren
- manche Kategorisierungen zwischen Funktionalität und Qualität unterscheiden, wie zum Beispiel IREB [IREB Foundation]
- Softwarearchitekt:innen die Qualitäten eines Softwaresystems beeinflussen können
- die Veränderung einer Qualität andere so beeinflussen kann, dass Abwägungen notwendig werden, wie z. B.
  - Konfigurierbarkeit versus Zuverlässigkeit
  - Speicherbedarf versus Leistungseffizienz
  - Sicherheit versus Benutzbarkeit
  - Laufzeitflexibilität versus Wartbarkeit.

Sie verstehen, dass sich eine einzelne Anforderung auf mehrere Qualitäten beziehen kann.

Referenzen

[IREB Foundation], [ISO 25010:2023], [Bass+2021], [Q42]

LZ 02-04: Anforderungen an Qualitäten formulieren (R1-R3)

Softwarearchitekt:innen:

- können Szenarien für gegebene Qualitäten mit Kontext, Stimulus, Reaktion und Messung für verschiedene Zwecke formulieren, z. B. um Anforderungen zu klären, Input für Architekturbewertungen zu liefern usw. (R1)
- verstehen, dass eine Anforderung für eine gegebene Qualität eine Analysemethode spezifizieren sollte (R1)
- wissen, dass die Verwendung einer Metrik als Ziel diese invalidieren kann (R2), wie z. B. in Goodhart’s Law (R3) beschrieben

Referenzen

[Bass+2021], [Q42], [ISO 25010:2023]

LZ 02-05: Explizite Aussagen vor impliziten Annahmen bevorzugen (R1, R2)

Softwarearchitekt:innen:

- können Annahmen oder Voraussetzungen explizit darstellen und dadurch implizite Annahmen vermeiden (R1)
- können erklären, warum implizite Annahmen zu Missverständnissen zwischen beteiligten Stakeholdern führen können (R1)
- können explizite Spezifikationen von Anforderungen, Randbedingungen, Grenzfällen, unzulässigen Aktionen und Eskalationspfaden schreiben, und behandeln dies als architektonische Verantwortlichkeit (R2)
- können erklären, warum gute Spezifikationen eindeutige, allgemeine Beschreibungen gewünschten Verhaltens enthalten - nicht nur Beispiele (R1)

3. Entwurf und Entwicklung von Softwarearchitekturen

Dauer: 270 Min.	Übungszeit: 90 Min.

Zielsetzung

Dieser Abschnitt zielt darauf ab, die Teilnehmenden in die Lage zu versetzen, Architekturentscheidungen so zu treffen, dass Anforderungen der Stakeholder unter Berücksichtigung der Randbedingungen erfüllt werden können. Sie lernen, Architekturentwürfe zu entwickeln, fundierte Entscheidungen zur Systemzerlegung zu treffen und Abhängigkeiten zwischen Bausteinen zu gestalten. Zu diesem Zweck lernen sie, grundlegende Ansätze und Heuristiken bei der Architekturentwicklung anzuwenden. Sie erkennen die Bedeutung von Entwurfsprinzipien und Lösungsmustern und können diese anwenden. Darüber hinaus werden in diesem Abschnitt der Umgang mit Querschnittsthemen, die Prinzipien des Software-Deployments und die Herausforderungen verteilter Systeme thematisiert.

Wesentliche Begriffe

Entwurf; Vorgehen beim Entwurf; Architekturentscheidung; Sichten; Schnittstellen; technische Konzepte und Querschnittsthemen; Muster; Mustersprachen; Entwurfsprinzipien; Abhängigkeiten; Kopplung; Kohäsion; Top-down- und Bottom-up-Vorgehen; modellbasierter Entwurf; iterativer Entwurf; Domain-Driven Design

Lernziele

LZ 03-01: Anforderungen durch Architektur erreichen (R1)

Softwarearchitekt:innen:

- verstehen, dass architektonische Aktivitäten davon geleitet werden sollten, spezifische Qualitäten zu erreichen oder zu verbessern
- können einen Architekturentwurf vorschlagen, der geeignet ist, Anforderungen zu erfüllen
- können einschätzen, welche Qualitäten sie durch bestimmte Aktivitäten oder Entscheidungen verbessern
- können mögliche Abwägungen zwischen Entwürfen sowie den entsprechenden Risiken identifizieren und kommunizieren

LZ 03-02: Softwarearchitekturen entwerfen (R1)

Softwarearchitekt:innen können:

- Softwarearchitekturen auf Basis bekannter funktionaler und Qualitätsanforderungen für nicht sicherheits- oder unternehmenskritische Softwaresysteme entwerfen und angemessen kommunizieren und dokumentieren
- Strukturentscheidungen hinsichtlich Systemzerlegung und Bausteinstruktur treffen, dabei Abhängigkeiten zwischen Bausteinen festlegen (siehe LZ 03-06: Abhängigkeiten von Bausteinen managen (R1))
- gegenseitige Abhängigkeiten und Abwägungen bezüglich Architekturentscheidungen erkennen und begründen
- Begriffe Blackbox und Whitebox erklären und zielgerichtet anwenden

- schrittweise Verfeinerung und Spezifikation von Bausteinen durchführen
- Architektursichten entwerfen, insbesondere Baustein-, Laufzeit- und Verteilungssicht (siehe LZ 04-05: Architektursichten erläutern und anwenden (R1))
- die aus Entscheidungen resultierenden Konsequenzen auf den Quellcode erklären
- domänenspezifische und technische Bestandteile in Architekturen trennen und diese Trennung begründen
- Risiken von Architekturentscheidungen identifizieren.

Referenzen

[Kruchten 1995], [Rozanski+2011], [Starke+2023a], [arc42], [Brown-C4], [Starke 2024]

LZ 03-03: Vorgehen und Heuristiken zur Architekturentwicklung auswählen und anwenden können (R1,R3)

Softwarearchitekt:innen können Vorgehensweisen, die Architekturentwicklung unterstützen, benennen erklären und anwenden, beispielsweise:

- Top-down- und Bottom-up-Vorgehen, siehe [Gharbi+2024], [Starke 2024] (R1)
- Architektursichten (R1)
- Domain-Driven Design, siehe [Evans 2004] (R3)
- Evolutionäre Architektur, siehe [Ford+2023] (R3)
- Quality-Driven Software Architecture, siehe [Starke 2021], [Bass+2021] (R3)
- Funktionale Softwarearchitektur, siehe [iSAQB FUNAR] (R3)

LZ 03-04: Entwurfsprinzipien erläutern und anwenden (R1-R3)

Softwarearchitekt:innen sind in der Lage zu erklären, was Entwurfsprinzipien sind. Sie können deren grundlegende Ziele und deren Anwendung im Hinblick auf Softwarearchitektur skizzieren. (R2)

Softwarearchitekt:innen sind in der Lage:

- die unten aufgeführten Gestaltungsprinzipien zu erläutern und mit Beispielen zu illustrieren
- zu erklären, wie diese Prinzipien angewendet werden sollen
- darzulegen, wie Anforderungen die Anwendung dieser Prinzipien beeinflussen
- die Auswirkungen der Entwurfsprinzipien auf die Implementierung zu erläutern
- Quellcode und Architektur zu analysieren, um zu beurteilen, ob diese Entwurfsprinzipien angewendet wurden oder angewendet werden sollten

Abstraktion (R2)

- im Sinne eines Vorgehens zur Erarbeitung zweckmäßiger Generalisierungen
- als eine Entwurfstechnik, bei dem die Bausteine von Abstraktionen und nicht von Implementierungen abhängen
- Schnittstellen als Abstraktionen

Modularisierung (R1)

- Geheimnisprinzip (Information Hiding) und Kapselung
- Trennung von Belangen (Separation of Concerns - SoC)
- Lose, aber funktionell ausreichende Kopplung von Bausteinen, siehe LZ 03-06: Abhängigkeiten von Bausteinen managen (R1)
- Hohe Kohäsion
- Offen/geschlossen-Prinzip
- Dependency-Inversion-Prinzip - Umkehrung von Abhängigkeiten durch Schnittstellen oder ähnliche Abstraktionen

Konzeptionelle Integrität (R2-R3)

- bedeutet Einheitlichkeit (Homogenität, Konsistenz) von Lösungen für ähnliche Probleme zu erreichen (R2)
- als ein Mittel, um das Prinzip der geringsten Überraschung zu erreichen (*principle of least surprise* oder *principle of least astonishment* (POLA)) (R3)
- Liskov'sches Substitutionsprinzip als eine Möglichkeit, Konsistenz, konzeptionelle Integrität und Robustheit (im Sinne von Typsicherheit) zu erreichen (R3).

Reduktion von Komplexität (R2-R3)

- als Motiv der Prinzipien KISS, YAGNI und CUPID [Terhorst-North 2022] (R3)
- DRY (Don't Repeat Yourself) als eine Möglichkeit, Wiederholungen zu vermeiden (R2)

Erwarte Fehler (R2-R3)

- als Mittel für den Entwurf robuster und widerstandsfähiger Systeme (R3)
- als eine Verallgemeinerung des Robustheitsgrundsatzes (*Postel's law*) (R2)

Make illegal states unrepresentable (R2)

- als ein Entwurfsprinzip, um die Robustheit verbessern

Referenzen

[Liskov+1994], [SOLID]

LZ 03-05: Zusammenhang zwischen Feedback-Schleifen und Risiko (R1, R2)

Softwarearchitekt:innen verstehen die Notwendigkeit von Iterationen, insbesondere bei unter Unsicherheit getroffenen Entscheidungen. Sie

- können den Einfluss von iterativem Vorgehen auf Architekturentscheidungen erläutern (hinsichtlich Risiken und Prognostizierbarkeit) (R1)
- können iterativ arbeiten und entscheiden (R1)
- verstehen die Notwendigkeit von Rückmeldungen zu Architekturentscheidungen (R1)
- können von anderen Stakeholdern systematisch Rückmeldung einholen. (R2)

LZ 03-06: Abhängigkeiten von Bausteinen managen (R1)

Softwarearchitekt:innen verstehen Abhängigkeiten und Kopplung zwischen Bausteinen und können diese gezielt einsetzen. Sie:

- kennen und verstehen unterschiedliche Arten der Kopplung von Bausteinen (beispielsweise Kopplung über Benutzung/Delegation, Nachrichten/Ereignisse, Komposition, Erzeugung, Vererbung, zeitliche Kopplung, Kopplung über Daten, Datentypen oder Hardware)
- verstehen, wie Abhängigkeiten die Kopplung vergrößern
- können mindestens die folgenden Arten von Kopplung unterscheiden:
  - statische und dynamische Kopplung
  - eingehende und ausgehende Abhängigkeiten
- wissen, dass auf statische Kopplung zu verzichten und stattdessen dynamische Kopplung einzusetzen, die zugrundeliegende Kopplung nicht notwendigerweise reduziert
- können Kopplung erkennen und ihre Auswirkungen bewerten
- können begründete Entscheidungen treffen, ob eine Abhängigkeit in Anbetracht der Anforderungen und Randbedingungen angemessen ist oder entfernt werden sollte
- kennen Möglichkeiten zur Auflösung bzw. Reduktion von Kopplung und können diese anwenden, beispielsweise:
  - Muster
  - Grundlegende Entwurfsprinzipien
  - Externalisierung von Abhängigkeiten, d.h. konkrete Abhängigkeiten erst zur Installations- oder Laufzeit festlegen, etwa durch Anwendung von Dependency Injection (R1) (siehe auch LZ 03-08: Wichtige Muster beschreiben, erklären und angemessen anwenden (R1, R3)).

Referenzen

[Ford+2021]

LZ 03-07: Schnittstellen entwerfen und spezifizieren (R1-R3)

Softwarearchitekt:innen kennen die kritische Bedeutung von Schnittstellen für die Interaktion zwischen Architekturbausteinen oder zwischen dem System und externen Elementen. Sie können solche Schnittstellen entwerfen und spezifizieren.

Sie kennen:

- wünschenswerte Eigenschaften von Schnittstellen und können diese beim Entwurf erreichen (R1):
  - einfach zu erlernen, einfach zu benutzen, einfach zu erweitern
  - schwer zu missbrauchen
  - funktional vollständig aus Sicht der Nutzer:innen oder nutzender Bausteine.
- die Notwendigkeit unterschiedlicher Behandlung interner und externer Schnittstellen (R2)
- die Trennung zwischen Schnittstelle und Implementierung (R1):
  - Implementierungen können bei Bedarf ausgetauscht werden.
- unterschiedliche Charakteristika von Schnittstellen, beispielsweise (R3):
  - Transportkanäle (etwa: TCP/IP als Teil des OSI 7-Schichten Modells)

    - intern oder extern
    - lokal oder remote
    - synchron oder asynchron
    - binär (nur maschinenlesbar) oder textuell (auch menschenlesbar)
    - zustandslos oder zustandsbehaftet
    - Punkt-zu-Punkt oder Multipunkt (broadcast oder multicast)
    - Funktionsaufruf (etwa: Remote Procedure Call) oder Nachrichtenaustausch
    - Batch, Request/Response oder Streaming
- Implementierungsansätze für synchrone Remote-Service-Schnittstellen, etwa (R3):
    - prozedurorientiert (etwa: GraphQL, WS-*/SOAP-basierte Webservices)
    - ressourcenorientiert (REST, REpresentational State Transfer)

Siehe auch LZ 04-06: Schnittstellen dokumentieren (R1).

Referenzen

[Zimmermann+2022], [Geewax 2021]

LZ 03-08: Wichtige Muster beschreiben, erklären und angemessen anwenden (R1, R3)

Softwarearchitekt:innen können die folgenden Muster erklären und Beispiele dafür liefern (R1):

- Schichten (Layers)
- Pipes und Filter
- Microservices
- Dependency Injection (siehe auch LZ 03-06: Abhängigkeiten von Bausteinen managen (R1))

Softwarearchitekt:innen können einige der folgenden Muster erklären, ihre Relevanz für konkrete Systeme erläutern und Beispiele dafür liefern (R3):

- Combinator
- Schnittstellenmuster wie Adapter, Facade, und Proxy. Architekt:innen sollten wissen, dass diese Muster unabhängig von bestimmten Programmiersprachen oder Frameworks verwendet werden können.
- Interpreter
- Observer
- Template Method und Strategy
- Visitor
- Blackboard
- Broker
- CQRS (Command-Query-Responsibility-Segregation)
- Event sourcing

- Integrations und Messaging-Patterns (z.B. aus [Hohpe+2004])
- Remote procedure call
- MVC (Model View Controller), MVVM (Model View ViewModel), MVU (Model View Update), PAC (Presentation Abstraction Control)
- Plugin
- Ports and Adapters (Synonyme: Onion Architektur, Hexagonal Architektur, Clean Architektur)
- SOA (Service-Oriented Architektur)

Softwarearchitekt:innen kennen wesentliche Quellen für Muster, beispielsweise GOF die POSA-Literatur (z. B. [Buschmann+1996]) und PoEAA ([Fowler 2003]) (für Informationssysteme). (R3)

Sie wissen:

- dass Muster ein Weg sind, bestimmte Qualitäten für gegebene Probleme und Anforderungen innerhalb gegebener Kontexte zu erreichen.
- zusätzliche Quellen für Muster, die sich auf ihre spezifische technische oder Anwendungsdomäne beziehen.

Referenzen

[Buschmann+1996], [Buschmann+2007], [Eilebrecht+2024], [Fowler 2003], [Gamma+1994], [Hohpe+2004], [Raj+2017]

LZ 03-09: Architektur analysieren (R1)

Softwarearchitekt:innen verstehen, dass eine Architekturanalyse helfen kann:

- Risiken und mögliche Verbesserungen im Architekturentwurf zu identifizieren
- festzustellen, ob der Architekturentwurf die Anforderungen erfüllt oder erfüllen wird
- zu erheben, inwiefern die Implementierung zu Architekturentscheidungen und Architekturentwurf passt
- zu überprüfen, inwiefern architekturrelevante Anliegen der Stakeholder berücksichtigt sind

Referenzen

LZ 03-10: Querschnittsthemen identifizieren und Querschnittskonzepte entwerfen und umsetzen (R1)

Softwarearchitekt:innen können:

- die Bedeutung von Querschnittsthemen erklären
- solche Querschnittsthemen identifizieren
- Querschnittskonzepte entwerfen, unter anderem Persistenz, Kommunikation, GUI, Fehlerbehandlung, Nebenläufigkeit, Energieeffizienz
- mögliche wechselseitige Abhängigkeiten erkennen und beurteilen.

Softwarearchitekt:innen wissen, dass solche Querschnittskonzepte systemübergreifend wiederverwendbar sein können.

Siehe auch LZ 04-07: Querschnittsthemen dokumentieren und kommunizieren (R2).

Referenzen

[Starke 2024], [Starke+2023a]

LZ 03-11: Grundlegende Prinzipien von Software-Deployments kennen (R3)

Softwarearchitekt:innen:

- wissen, dass Software-Deployment der Prozess ist, durch den neue oder aktualisierte Software zur Benutzung bereitgestellt wird
- können grundlegende Konzepte des Deployments von Software benennen und erklären:
  - Automatisierung von Deployments
  - Wiederholbare Builds
  - Konsistente Umgebungen (z. B. durch Nutzung von unveränderlicher (*immutable*) Infrastruktur)
  - Alles liegt unter Versionskontrolle
  - Releases sind einfach zurückzunehmen

Referenzen

[Humble+2010]

LZ 03-12: Technische und geschäftliche Sichten verwenden können (R3)

Softwarearchitekt:innen:

4. Beschreibung und Kommunikation von Softwarearchitekturen

Dauer: 180 Min.	Übungszeit: 60 Min.

Zielsetzung

Ziel dieses Abschnitts ist es, die Teilnehmer:innen in die Lage zu versetzen, Softwarearchitekturen so zu dokumentieren und zu kommunizieren, dass sie den Bedürfnissen wichtiger Stakeholder gerecht werden und den Entwicklungsprozess unterstützen. Der Schwerpunkt liegt auf dem Verständnis der grundlegenden Anforderungen an technische Dokumentationen, der Verwendung geeigneter Modelle und Notationen zur Beschreibung von Architekturen und der Anwendung wichtiger Architekturansichten. Außerdem lernen die Teilnehmer:innen, wichtige Architekturentscheidungen, Schnittstellen und Querschnittskonzepte zu dokumentieren, um eine klare, korrekte und für die Stakeholder relevante Dokumentation zu gewährleisten.

Wesentliche Begriffe

(Architektur-)Sichten; Strukturen; (technische) Konzepte; Dokumentation; Kommunikation; Beschreibung; Ausrichtung an Zielgruppen und Stakeholder-Anliegen; Meta-Strukturen und Templates zur Beschreibung und Kommunikation; Kontextabgrenzung; Bausteine; Bausteinsicht; Laufzeitsicht; Verteilungssicht; Knoten; Kanal; Verteilungsartefakte; Mapping von Bausteinen auf Verteilungsartefakte; Mapping von Verteilungsartefakten auf Knoten; Beschreibung von Schnittstellen und Architekturentscheidungen; UML; Werkzeuge zur Dokumentation

Lernziele

LZ 04-01: Anforderungen an technische Dokumentation erläutern und berücksichtigen (R1)

Softwarearchitekt:innen kennen die wesentlichen Anforderungen an technische Dokumentation und können diese bei der Dokumentation von Systemen berücksichtigen bzw. erfüllen:

- Verständlichkeit, Korrektheit, Effizienz, Angemessenheit, Wartbarkeit
- Orientierung von Form, Inhalt und Detailgrad an Zielgruppe der Dokumentation

Sie wissen, dass Verständlichkeit technischer Dokumentation nur von deren Zielgruppen beurteilt werden kann.

Referenzen

[Starke+2023a], [Zörner 2021], [Clements+2010]

LZ 04-02: Softwarearchitekturen beschreiben und kommunizieren (R1-R3)

Softwarearchitekt:innen nutzen Dokumentation zur Unterstützung bei Entwurf, Implementierung und Weiterentwicklung (auch genannt *Wartung* oder *Evolution*) von Systemen. (R2)

Softwarearchitekt:innen können (R1):

- Architekturen entsprechend der Anliegen der Stakeholder dokumentieren und kommunizieren und dadurch unterschiedliche Zielgruppen adressieren, z. B. Management, Entwicklungsteams, QS, andere Softwarearchitekt:innen sowie möglicherweise zusätzliche Stakeholder
- Maßnahmen entwickeln und umsetzen, die mündliche und schriftliche Kommunikation in Einklang

miteinander halten und miteinander angemessen ausbalancieren

- den Nutzen von Template-basierter Dokumentation erklären
- beschreiben, wie verschiedene Eigenschaften der Dokumentation von Spezifika des Systems, seinen Anforderungen, Risiken, dem Entwicklungsvorgehen, der Organisation oder anderen Faktoren abhängen.

Beispielsweise können Softwarearchitekt:innen die Dokumentation situationsbezogen in Umfang, Detaillierungsgrad, Format sowie Zugänglichkeit anpassen.

LZ 04-03: Notations-/Modellierungsmittel für Beschreibung von Softwarearchitektur erläutern und anwenden (R2-R3)

Softwarearchitekt:innen können für gegebene Stakeholder-Anliegen eine angemessene Notation auswählen und ihre Wahl begründen. Sie berücksichtigen dabei Zielgruppe, benötigten Detaillierungsgrad und Formalitätsgrad, siehe LZ 04-01: Anforderungen an technische Dokumentation erläutern und berücksichtigen (R1) und LZ 04-02: Softwarearchitekturen beschreiben und kommunizieren (R1-R3) (R2).

Insbesondere können sie begründen, wann eine textuelle oder tabellarische Darstellung (z. B. nummerierte Ablaufschritte, Schnittstellentabelle) einem Diagramm vorzuziehen ist (R2).

Softwarearchitekt:innen können folgende UML-Diagrammtypen *lesen*, also die dargestellten Elemente, Beziehungen und deren Semantik erklären, und für einfache Systeme *erstellen* (R2):

- Baustein- und Kontextsicht: Paket-, Komponenten- und Klassendiagramme
- Laufzeitsicht: Sequenz- und Aktivitätsdiagramme
- Verteilungssicht: Verteilungsdiagramme

Beim Erstellen achten sie mindestens auf: eindeutige Zweckangabe, Legende bzw. Erklärung nicht-standardkonformer Elemente, passenden Abstraktionsgrad und Konsistenz zum begleitenden Text. Beim Lesen erkennen sie Verstöße gegen diese Kriterien. (R2)

Softwarearchitekt:innen kennen weitere Notationen und deren primären Einsatzzweck: Zustandsdiagramme und Composite-Structure-Diagramme (UML), C4 [Brown-C4], ArchiMate, SysML, Entity-Relationship-Diagramme [Chen 1976], Flussdiagramme, BPMN (R3).

Referenzen

[UML], [ArchiMate], [SysML], [Brown-C4], [Chen 1976]

LZ 04-04: Lernziel nicht gefunden (R3)

Softwarearchitekt:innen können unerwartete Situationen geschickt bewältigen.

Hinweis: Der Titel dieses Lernziels wurde bewusst gewählt und stellt keinen technischen Fehler dar.

Referenzen

[IETF HTTP]

LZ 04-05: Architektursichten erläutern und anwenden (R1)

Softwarearchitekt:innen können folgende Architektursichten anwenden:

- Kontextsicht (auch genannt Kontextabgrenzung)
  - enthält die externen Schnittstellen von Systemen
  - bei Bedarf differenziert nach fachlichem und technischem Kontext
- Baustein- oder Komponentensicht (Aufbau des Systems aus Softwarebausteinen)
- Laufzeitsicht (dynamische Sicht, Zusammenwirken der Softwarebausteine zur Laufzeit, Zustandsmodelle)
- Verteilungs-/Deploymentsicht (Hardware und technische Infrastruktur sowie Abbildung von Softwarebausteinen auf diese Infrastruktur)

Zusätzliche Sichten können nach Bedarf verwendet werden, um weitere Anliegen oder Anforderungen von Stakeholdern zu berücksichtigen. (R3)

Referenzen

[Kruchten 1995], [Rozanski+2011], [Starke+2023a], [arc42], [Brown-C4]

LZ 04-06: Schnittstellen dokumentieren (R1)

Softwarearchitekt:innen können sowohl interne als auch externe Schnittstellen dokumentieren.

Siehe auch LZ 03-07: Schnittstellen entwerfen und spezifizieren (R1-R3).

LZ 04-07: Querschnittsthemen dokumentieren und kommunizieren (R2)

Softwarearchitekt:innen können typische Querschnittsthemen und die dazugehörigen Lösungskonzepte (Querschnittskonzepte) adäquat dokumentieren und kommunizieren, z. B. Persistenz, Ablaufsteuerung, UI, Verteilung/Integration, Protokollierung.

Siehe auch LZ 03-10: Querschnittsthemen identifizieren und Querschnittskonzepte entwerfen und umsetzen (R1).

LZ 04-08: Architekturentscheidungen erläutern und dokumentieren (R1-R2)

Softwarearchitekt:innen können:

- Architekturentscheidungen systematisch herbeiführen, begründen, kommunizieren und dokumentieren
- gegenseitige Abhängigkeiten solcher Entscheidungen erkennen, kommunizieren und dokumentieren

Softwarearchitekt:innen kennen Architecture-Decision-Records (ADR, siehe [Nygard 2011]) und können diese zur Dokumentation von Entscheidungen einsetzen (R2).

Referenzen

[Nygard 2011]

LZ 04-09: Weitere Hilfsmittel und Werkzeuge zur Dokumentation kennen (R2)

Softwarearchitekt:innen kennen:

- Grundlagen mehrerer publizierter Frameworks zur Beschreibung von Softwarearchitekturen, beispielsweise:

    - arc42,
    - C4, siehe [Brown-C4]
- Ideen und Beispiele von Checklisten für die Erstellung, Dokumentation und Prüfung von Softwarearchitekturen
- mögliche Werkzeuge zur Erstellung und Pflege von Architekturdokumentation

Referenzen

[ISO 42010:2022], [arc42], [Brown-C4]

5. Einsatz und Risiken von Künstlicher Intelligenz

Dauer: 60 Min.	Übungszeit: 0 Min.

Zielsetzung

Ziel dieses Abschnitts ist es, den Teilnehmer:innen ein grundlegendes Verständnis der Verwendung künstlicher Intelligenz zur Unterstützung von Architekturarbeit zu vermitteln. Der Abschnitt deckt den Einfluss von KI-generierten Artefakten auf die Qualität ab und die Wichtigkeit, KI-generierten Output zu verifizieren, sowie die systemischen Risiken von KI und ihre ethischen und gesellschaftlichen Externalitäten. Eine umfangreichere Abdeckung von KI-unterstützter Architektur gibt es im Advanced-Level-Modul AGENTA [iSAQB AGENTA]. Die Architektur von KI-basierten System wird vom Advanced-Level-Modul SWARC4AI [iSAQB SWARC4AI] abgedeckt.

Wesentliche Begriffe

LLM, KI-Agent (z. B. Coding-Agent), KI-generierter Output

Lernziele

LZ 05-01: Über den Einsatz von KI in der Architekturarbeit entscheiden (R1, R2)

Softwarearchitekt:innen:

- können anhand benannter Kriterien entscheiden, ob und wie KI für eine gegebene Aufgabe eingesetzt wird, und dies als Architekturentscheidung begründen (R2)
- können erklären, warum beim Delegieren von Aufgaben an KI-Agenten die Konsequenzen impliziter Annahmen verstärkt werden: Agenten können auf mehrdeutige, unvollständige oder widersprüchliche Anweisungen auf eine Art reagieren, wie es Menschen nicht tun würden (R1)

LZ 05-02: Verifikation von KI-generierten Artefakten (R1, R2)

Softwarearchitekt:innen:

- können erklären, dass die Verantwortung für Entscheidungen und Artefakte bei Menschen verbleibt, auch wenn diese teilweise KI-generiert sind (R1)
- können erklären, warum die Verifikation von KI-generiertem Output integraler Bestandteil des Entwicklungsprozesses sein muss, da solcher Output auf nicht-offensichtliche Weise inkorrekt sein kann (R1)
- können Verifikationsansätze benennen, z. B. Code-Reviews, automatisierte Tests und Continuous Integration, statische Analyse und Typsysteme, szenariobasierte und formale Methoden (R2)
- können die Risiken erklären, Review und Verifikation an ein LLM zu delegieren (R2)

Referenzen

[iSAQB FM]

LZ 05-03: Risiken, ethische und gesellschaftliche Externalitäten erkennen (R2, R3)

Softwarearchitekt:innen:

- können Risiken des KI-Einsatzes benennen: rechtliche Risiken bezüglich des geistigen Eigentums, Offenlegung von Quellcode, Dokumentation oder Daten gegenüber externen Modellanbietern, Sicherheitsrisiken wie Prompt Injection, Abbau kritischer Fähigkeiten und Störung von Lernprozessen sowie Abhängigkeiten von einer kleinen Gruppe von Anbietern und von bestimmten Modellen, die verändert oder abgekündigt werden können (R2)
- können die gesellschaftlichen Externalitäten von KI-Infrastruktur beschreiben: Energie/CO_2, Druck auf den Arbeitsmarkt, Disruption von Open-Source- und Internet-Ökosystemen, Modell-Bias in öffentlicher Infrastruktur, politische Destabilisierung (R3)

Referenzen

[Link 2025]

6. Beispiele für Softwarearchitekturen

Dauer: 90 Min.	Übungszeit: Keine

Dieser Abschnitt MUSS in der Schulung behandelt werden (z. B. anhand eines konkreten praktischen Beispiels), ist jedoch nicht prüfungsrelevant.

Lernziele

LZ 06-01: Bezug von Anforderungen und Randbedingungen zur Lösung erfassen (R3)

Softwarearchitekt:innen haben an mindestens einem Beispiel den Bezug von Anforderungen und Randbedingungen zu Lösungsentscheidungen erkannt und nachvollzogen.

Referenzen

[arc42], [Starke+2023b], [Hruschka+2021], [Zörner 2021], [AOSA]

LZ 06-02: Technische Umsetzung einer Lösung nachvollziehen (R3)

Softwarearchitekt:innen können anhand mindestens eines Beispiels die technische Umsetzung (Implementierung, technische Konzepte, eingesetzte Produkte, Lösungsstrategien) einer Lösung nachvollziehen.

Referenzen

[arc42], [Starke+2023b], [Hruschka+2021], [Zörner 2021], [AOSA]

Referenzen

- [AOSA] A. Brown and G. Wilson, Eds., *The Architecture of Open Source Applications*. [Online]. Available: https://aosabook.org/en/
- [arc42] "arc42, the open-source template for software architecture communication." [Online]. Available: https://arc42.org, maintained on https://github.com/arc42.
- [ArchiMate] "The ArchiMate® Enterprise Architecture Modeling Language." [Online]. Available: https://www.opengroup.org/archimate-forum/archimate-overview
- [Bass+2021] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison Wesley, 2021.
- [Brown-C4] S. Brown, "The C4 model for visualising software architecture." [Online]. Available: https://c4model.com, https://www.infoq.com/articles/C4-architecture-model
- [IREB Foundation] S. Bühne, M. Glinz, H. van Loen, and S. Staal, *Certified Professional for Requirements Engineering - Foundation Level - Syllabus*, Version 3.2.0, IREB, 2024.
- [Buschmann+1996] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture (POSA): A System of Patterns*, Wiley, 1996.
- [Buschmann+2007] F. Buschmann, K. Henney, and D. C. Schmidt, *Pattern-Oriented Software Architecture (POSA): A Pattern Language for Distributed Computing*, Wiley, 2007.
- [Clements+2002] P. Clements, R. Kazman, and M. Klein, *Evaluating Software Architectures: Methods and Case Studies*, Addison Wesley, 2002.
- [Clements+2010] P. Clements et al., *Documenting Software Architectures: Views and Beyond*, 2nd ed., Addison Wesley, 2010.
- [CNCF] The Cloud Native Computing Foundation. [Online]. Available: https://www.cncf.io/
- [Eilebrecht+2024] K. Eilebrecht and G. Starke, *Patterns kompakt: Entwurfsmuster für effektive Software-Entwicklung* (in German), 6th ed., Springer Verlag, 2024.
- [Chen 1976] P. Chen, "The Entity-Relationship Model: Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976.
- [Evans 2004] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2004.
- [Felleisen+2014] M. Felleisen, R. B. Findler, M. Flatt, and S. Krishnamurthi, *How to Design Programs*, 2nd ed., MIT Press, 2014. [Online]. Available: https://htdp.org/
- [Ford+2023] N. Ford, R. Parsons, P. Kua, and P. Sadalage, *Building Evolutionary Architectures: Automated Software Governance*, 2nd ed., O'Reilly Media, 2023.
- [Ford+2021] N. Ford, M. Richards, P. Sadalage, and Z. Dehghani, *Software Architecture: The Hard Parts*. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [Fowler 2003] M. Fowler, *Patterns of Enterprise Application Architecture* (PoEAA), Addison-Wesley, 2003.
- [Gandhi+2024] R. Gandhi, M. Richards, and N. Ford, *Head-First Software Architecture*, O'Reilly, 2024.
- [Gamma+1994] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

- [Geewax 2021] J. Geewax, *API Design Patterns*, Manning, 2021. This book lays out a set of design principles for building internal and public-facing APIs.
- [Geirhos 2015] M. Geirhos, *Entwurfsmuster: Das umfassende Handbuch* (in German), Rheinwerk Computing Verlag, 2015.
- [Gharbi+2024] M. Gharbi, A. Koschel, A. Rausch, and G. Starke, *Basiswissen Softwarearchitektur* (in German), 5th ed. Heidelberg: dpunkt Verlag, 2024.
- [Goll 2014] J. Goll, *Architektur- und Entwurfsmuster der Softwaretechnik: Mit lauffähigen Beispielen in Java* (in German), 2nd ed., Springer-Vieweg Verlag, 2014.
- [Hohpe+2004] G. Hohpe and B. A. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*, Addison-Wesley Professional, 2004.
- [Hombergs 2024] T. Hombergs, *Get Your Hands Dirty on Clean Architecture*, 2nd ed., Packt, 2024.
- [Humble+2010] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Pearson International, 2010.
- [Hruschka+2021] P. Hruschka, I. Kostov, and W. Reimesch, *arc42 by Example - Volume 2: Architecture Documentation for Embedded Systems and IoT*, Leanpub, 2021. [Online]. Available: https://leanpub.com/arc42byexample-volume2
- [IETF HTTP] Internet Engineering Task Force, "HTTP Semantics," RFC 9110. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html
- [iSAQB AGENTA] iSAQB e.V., *Curriculum for Certified Professional for Software Architecture (CPSA) Advanced Level: AGENTA - Architecture for Agentic Software Engineering Contexts*, iSAQB e.V., 2026. [Online]. Available: https://public.isaqb.org/curriculum-agenta/
- [iSAQB Downloads] "iSAQB public download site." [Online]. Available: https://public.isaqb.org. Contains curricula and mock-examination.
- [iSAQB FM] iSAQB e.V., *Curriculum for Certified Professional for Software Architecture (CPSA) Advanced Level: FM - Formal Methods*, iSAQB e.V., 2024. [Online]. Available: https://public.isaqb.org/curriculum-fm/
- [iSAQB FUNAR] iSAQB e.V., *Curriculum for Certified Professional for Software Architecture (CPSA) Advanced Level: FUNAR - Functional Software Architecture*, iSAQB e.V., 2023. [Online]. Available: https://public.isaqb.org/curriculum-funar/
- [iSAQB Glossary] G. Starke et al., *iSAQB Glossary of Software Architecture Terminology*, Leanpub, 2020-2026. [Online]. https://github.com/isaqb-org/glossary/releases
- [iSAQB SWARC4AI] iSAQB e.V., *Curriculum for Certified Professional for Software Architecture (CPSA) Advanced Level: SWARC4AI - Software Architecture for AI Systems*, iSAQB e.V., 2026. [Online]. Available: https://public.isaqb.org/curriculum-swarc4ai/
- [ISO 25010:2023] *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, ISO/IEC 25010:2023, 2023. [Online]. Available: https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-2:v1:en (terms and definitions)
- [ISO 25019:2023] *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality-in-use model*, ISO/IEC 25019:2023, 2023. [Online]. Available: https://www.iso.org/obp/ui/#iso:std:iso-iec:25019:ed-1:v1:en (terms and definitions)
- [ISO 42010:2022] *Software, systems and enterprise Architecture description*, ISO/IEC/IEEE 42010:2022, 2022. [Online]. Available: https://www.iso.org/standard/74393.html

- [Keeling 2017] M. Keeling, *Design It!: From Programmer to Software Architect*, Pragmatic Bookshelf, 2017.
- [Kleppmann 2017] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.
- [Kruchten 1995] P. Kruchten, "Architectural Blueprints—The '4+1' View Model of Software Architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, Nov. 1995.
- [Kruchten 2004] P. Kruchten, *The Rational Unified Process: An Introduction*, 3rd ed., Addison-Wesley Professional, 2004.
- [Lange 2021] K. Lange, "The Functional Core, Imperative Shell Pattern," Mar. 27, 2021. [Online]. Available: https://www.kennethlange.com/functional-core-imperative-shell/
- [Lehman 1980] M. M. Lehman, "Programs, Life Cycles, and Laws of Software Evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.
- [Wiki-LehmansLaws] "Lehman's laws of software evolution," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Lehman%27s_laws_of_software_evolution
- [Lilienthal 2024] C. Lilienthal, *Langlebige Softwarearchitekturen* (in German), 4th ed., dpunkt Verlag, 2024.
- [Lilienthal 2019] C. Lilienthal, *Sustainable Software Architecture: Analyze and Reduce Technical Debt*, dpunkt Verlag, 2019.
- [Link 2025] J. Link, "To Gen or Not To Gen: The Ethical Use of Generative AI," Nov. 4, 2025. [Online]. Available: https://blog.johanneslink.net/2025/11/04/to-gen-or-not-to-gen/
- [Liskov+1994] B. H. Liskov and J. M. Wing, "A behavioral notion of subtyping," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 6, 1994, doi: 10.1145/197320.197383.
- [Maguire 2023] S. Maguire, *Algebra-Driven Design: Elegant Solutions from Simple Building Blocks*, Leanpub, 2023. [Online]. Available: https://leanpub.com/algebra-driven-design
- [Newman 2021] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed., O'Reilly, 2021.
- [Terhorst-North 2022] D. Terhorst-North, "CUPID: for joyful coding." [Online]. Available: https://dannorth.net/2022/02/10/cupid-for-joyful-coding/
- [Nygard 2011] M. Nygard, "Documenting Architecture Decisions." [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. See also https://adr.github.io/
- [Raj+2017] P. Raj, A. Raman, and H. Subramanian, *Architectural Patterns: Uncover Essential Patterns in the Most Indispensable Realm of Enterprise Architecture*, Packt Publishing, 2017.
- [Pohl 2025] K. Pohl, *Requirements Engineering: Fundamentals, Principles and Techniques*, 2nd ed., Springer, 2025.
- [Q42] "arc42 Quality Model." [Online]. Available: https://quality.arc42.org. Pragmatic (and open-source) alternative to [ISO 25010:2023].
- [Rajlich+2000] V. T. Rajlich and K. H. Bennett, "A Staged Model for the Software Life Cycle," *IEEE Computer*, vol. 33, no. 7, pp. 66–71, 2000.
- [Read 2023] J. Read, *Communication Patterns: A Guide for Developers and Architects*, O'Reilly, 2023.

- [Richards+2020] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*, O'Reilly, 2020.
- [Rozanski+2011] N. Rozanski and E. Woods, *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives*, 2nd ed., Addison-Wesley, 2011.
- [SOLID] S. Oloruntoba and A. S. Walia, "SOLID: The First 5 Principles of Object Oriented Design." [Online]. Available: https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design
- [Sperber+2023] M. Sperber and H. Klaeren, *Schreibe Dein Programm!* (in German), Tübingen University Press, 2023. [Online]. Available: https://www.deinprogramm.de/sdp/
- [Sperber+2024] M. Sperber and S. Wehr, "Datenmodellierung mit Summen und Produkten" (in German), 2024. [Online]. Available: https://funktionale-programmierung.de/2024/11/25/sums-products.html (English translation: Data Modeling with Sums and Products, 2024. https://funktionale-programmierung.de/2024/11/25/sums-products-english.html)
- [Starke 2024] G. Starke, *Effektive Software-Architekturen: Ein praktischer Leitfaden* (in German), 10th ed. Munich, Germany: Carl Hanser Verlag, 2024. Website: https://esabuch.de
- [Starke 2021] G. Starke, "Quality Driven Software Architecture - Revised," 2021. [Online]. Available: https://www.innoq.com/en/blog/2021/08/quality-driven-software-architecture-revised/
- [Starke+2023a] G. Starke and A. Lorz, *Software Architecture Foundation: CPSA Foundation® Exam Preparation*, 2nd ed., Van Haren Publishing, 2023.
- [Starke+2023b] G. Starke, M. Simons, S. Zörner, R. D. Müller, and H. Lösch, *arc42-by-Example: Software Architecture Documentation in Practice*, 3rd ed., Leanpub, 2023. [Online]. Available: https://leanpub.com/arc42byexample
- [SysML] "What is SysML." [Online]. Available: https://sysml.org/. For diagrams, see also https://sysml.org/tutorials/sysml-diagram-tutorial/.
- [UML] "The UML reading room," collection of UML resources. [Online]. Available: https://www.omg.org/technology/readingroom/UML.htm. See also https://www.uml-diagrams.org/.
- [Yorgey 2012] B. A. Yorgey, "Monoids: Theme and Variations," in *Proc. 2012 Haskell Symposium*, Sep. 2012, doi: https://doi.org/10.1145/2364506.2364520
- [Zimmermann+2022] O. Zimmermann, M. Stocker, D. Lübke, U. Zdun, and C. Pautasso, *Patterns for API Design: Simplifying Integration with Loosely Coupled Message Exchanges*, Addison-Wesley, 2022.
- [Zörner 2021] S. Zörner, *Softwarearchitekturen dokumentieren und kommunizieren* (in German), 3rd ed., Carl Hanser Verlag, 2021.